

LAB 10.1 – AI Assisted Coding

Task 1 – Syntax and Logic Errors

Question: Use AI to identify and fix syntax and logic errors in a faulty Python script.

Corrected Code:

```
def calc_average(marks: list) -> float:
    total = sum(marks)
    average = total / len(marks)
    return average

marks = [85, 90, 78, 92]
print("Average Score is ", calc_average(marks))
```

Output:

Average Score is 86.25

Task 2 – PEP 8 Compliance

Question: Use AI to refactor Python code to follow PEP 8 style guidelines.

Refactored Code:

```
def area_of_rect(length: float, breadth: float) -> float:
    return length * breadth

print(area_of_rect(10, 20))
```

Output:

200

Task 3 – Readability Enhancement

Question: Use AI to make code more readable without changing its logic.

Improved Code:

```
def calculate_percentage(value: float, percentage: float) -> float:
    return value * percentage / 100
```

```
a = 200
b = 15
result = calculate_percentage(a, b)
print(f"{b}% of {a} is {result}")
```

Output:

15% of 200 is 30.0

Task 4 – Refactoring for Maintainability

Question: Use AI to break repetitive or long code into reusable functions.

Refactored Code:

```
def welcome_students(students: list) -> None:
    for student in students:
        print("Welcome", student)
```

```
students = ["Alice", "Bob", "Charlie"]
welcome_students(students)
```

Output:

Welcome Alice
Welcome Bob
Welcome Charlie

Task 5 – Performance Optimization

Question: Use AI to make the code run faster.

Optimized Code:

```
nums = range(1, 1000000)
squares = [n**2 for n in nums]
print(len(squares))
```

Output:

999999

Task 6 – Complexity Reduction

Question: Use AI to simplify overly complex logic.

Simplified Code:

```
def grade(score: float) -> str:
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 60:
        return "D"
    else:
        return "F"

print(grade(85))
```

Output:

B